

Relatorio completo de dados e banco de dados - EN Met@

Data da avaliacao: 10/06/2026 Ambiente avaliado: maquina local Windows em `C:\Users\igor\Documents\enterall-smart-rx` Banco avaliado: PostgreSQL local Docker, database `enterall_smart_rx` Backend avaliado: Express + Prisma em `http://localhost:3000` Frontend avaliado: Vite/React em `http://localhost:8080` Tunel publico atual: `https://shipments-posts-hottest-grab.trycloudflare.com`

1. Resumo executivo

O EN Met@ esta estruturado como um sistema hospitalar local-first com:

- frontend React/Vite;
- backend Node/Express;
- banco PostgreSQL local;
- Prisma como camada de acesso ao banco;
- fila offline no navegador via IndexedDB/Dexie;
- observabilidade com Prometheus, Grafana e metricas do backend.

Tecnicamente, a estrutura do banco esta correta e sincronizada com o Prisma. A stack local esta no ar, e o backend respondeu `ready` com banco `ok`.

Atualizacao da analise: a leitura anterior de banco vazio estava incorreta porque se baseou em estatisticas estimadas do PostgreSQL (`pg_stat_user_tables`), que podem ficar desatualizadas. A contagem real feita pelo Prisma, usando a mesma conexao do backend, confirma que o PostgreSQL local contem dados.

Contagem real encontrada em 10/06/2026:

- 2 hospitais;
- 6 alas/setores;
- 6 profissionais/usuarios;
- 17 pacientes;
- 32 formulas;
- 11 modulos;
- 7 insumos;
- 20 prescricoes;
- 59 evolucoes/acompanhamentos diarios;
- 377 registros de idempotencia da sincronizacao.

Conclusao operacional corrigida: o app esta salvando no PostgreSQL quando o backend esta online. Quando o navegador esta offline ou o servidor local esta indisponivel, as operacoes ficam na fila offline do navegador, em IndexedDB/Dexie, e depois sao reenviadas para o backend quando a conexao volta.

Isso significa que existem duas camadas de persistencia em uso:

- persistencia principal: PostgreSQL local em Docker, acessado pelo backend via Prisma;
- persistencia temporaria/offline: IndexedDB no navegador, banco `enmeta-offline`.

Tambem ha uso de `localStorage` para sessao, unidade selecionada e alguns fallbacks historicos, mas ele nao deve ser tratado como banco principal.

2. Estado dos servicos locais

Servicos validados em 10/06/2026:

Servico	URL/porta	Status
PostgreSQL	localhost:5432	container healthy
Backend	http://localhost:3000/health/ready	200 , database ok
Frontend	http://localhost:8080	200
Grafana	http://localhost:3001/api/health	200
Prometheus	http://localhost:9090/-/healthy	200
Tunnel Cloudflare	https://shipments-posts-hottest-grab.trycloudflare.com/health/ready	200 , database ok

O arquivo `vercel.json` foi atualizado para redirecionar:

- `/health` para o tunnel novo;
- `/health/:path*` para o tunnel novo;
- `/api/:path*` para o tunnel novo.

Isso faz o frontend publicado na Vercel chamar o backend local por meio do Cloudflare Tunnel.

3. Como os dados chegam ao banco

O fluxo principal e:

1. Usuario interage com a tela no frontend React. 2. A tela chama um hook em `src/hooks/useDatabase.ts`. 3. O hook chama um service em `src/lib/database.ts`. 4. O service chama `apiClient` em `src/lib/api.ts`. 5. `apiClient` envia HTTP para `/api/...`. 6. Em desenvolvimento local, `/api` vai para `http://localhost:3000/api`. 7. Em producao Vercel, `/api` e mesmo dominio da Vercel, mas `vercel.json` reescreve para o tunnel Cloudflare. 8. O backend Express recebe a requisicao em `server/src/routes/...`. 9. O backend valida JWT/permissoes/hospital quando necessario. 10. O backend usa Prisma para gravar/ler PostgreSQL. 11. O PostgreSQL persiste os dados no volume Docker `postgres_postgres_data`, montado no container em `/var/lib/postgresql/data`.

Fluxo resumido:

```
Tela React
-> useDatabase.ts
-> database.ts
-> api.ts
-> /api/... no backend
-> Express route
-> Prisma Client
-> PostgreSQL
```

4. Como os dados chegam quando esta offline

O app tem uma camada offline em `src/lib/offlineStore.ts`.

Banco local do navegador:

```
IndexedDB: enmeta-offline
```

Tabelas Dexie dentro do navegador:

Store IndexedDB	Funcao
<code>pendingOperations</code>	fila de criacoes, atualizacoes e exclusoes aguardando envio ao servidor
<code>snapshots</code>	copia local de listas recebidas do backend
<code>shadowRecords</code>	versao local editada/criada/deletada antes de sincronizar
<code>idMappings</code>	mapeia IDs temporarios locais para IDs reais do backend

Quando o usuario cria ou edita algo:

- se a API responde, salva no backend e atualiza o snapshot local;
- se o navegador esta offline ou o servidor cai, a operacao vai para `pendingOperations` ;
- o app cria ou atualiza `shadowRecords` , para a interface ja mostrar a alteracao;
- quando volta a conexao, `flushPendingOperations()` tenta reenviar a fila;
- cada operacao usa `x-idempotency-key` e `x-device-id` .

Esse desenho e bom porque evita perder trabalho do usuario quando o problema e rede/servidor temporariamente indisponivel.

Pontos de atencao:

- Se o erro for 400/401/403/404 por problema de dado, permissao ou referencia inexistente, a operacao pode ficar `failed` .
- Se houver conflito de versao 409, a fila exige decisao da Central Sync.
- Se o navegador for formatado antes de sincronizar, a fila offline some.
- Por isso, o backup oficial precisa ser do PostgreSQL, nao apenas do navegador.

5. Estado atual do PostgreSQL

Banco:

```
Database: enterall_smart_rx
Tamanho atual aproximado: 9823 kB
Schema Prisma: sincronizado
Tabelas de negocio: 17
Registros atuais: 0 em todas as tabelas de negocio
```

Tamanho fisico por tabela, mesmo vazias, por causa de estrutura, indices e metadados:

Tabela	Tamanho aproximado
IdempotencyRecord	720 kB
Prescription	240 kB
DailyEvolution	128 kB
Formula	112 kB
Patient	80 kB
Supply	80 kB
Professional	64 kB
AppSettings	64 kB
Ward	48 kB
Module	48 kB
PrescriptionStatusEvent	48 kB
PrescriptionFormula	32 kB
PrescriptionModule	32 kB
Hospital	32 kB
AppTool	24 kB
RolePermission	24 kB
PrescriptionSupply	16 kB

Contagem real atual, validada via Prisma:

Tabela	Registros
Hospital	2
Ward	6
Professional	6
Patient	17
Formula	32
Module	11
Supply	7
Prescription	20
PrescriptionFormula	13
PrescriptionModule	10
PrescriptionSupply	0
PrescriptionStatusEvent	22
DailyEvolution	59
AppSettings	2
RolePermission	0
AppTool	0
IdempotencyRecord	377

Conclusao operacional: o banco esta tecnicamente funcionando e contem dados reais/demonstrativos suficientes para o app operar localmente. As tabelas `RolePermission` e `AppTool` estao vazias porque a matriz de permissoes e as ferramentas ainda podem estar vindo de padroes do codigo ou configuracoes locais. A tabela `PrescriptionSupply` esta vazia porque os insumos parecem ser calculados/derivados nos relatorios e etiquetas, nao necessariamente gravados como itens persistidos em toda prescricao.

6. Tabelas do banco e finalidade

6.1 Hospital

Tabela mestre da unidade hospitalar.

Colunas:

- `id` : identificador UUID/texto.
- `name` : nome do hospital.
- `cnpj` : CNPJ.
- `address` : endereco.
- `city` : cidade.
- `state` : UF.
- `zipCode` : CEP.
- `phone` : telefone.
- `email` : email.
- `isActive` : unidade ativa/inativa.
- `version` : controle de versao para sincronizacao/conflito.
- `createdAt` : criacao.

- `updatedAt` : ultima atualizacao.

Relacionamentos:

- possui pacientes, formulas, modulos, insumos, profissionais, prescricoes, evolucoes, alas, configuracoes, permissoes e ferramentas.

Organizacao recomendada:

- criar uma linha por unidade real;
- nao duplicar hospitais por erro de grafia;
- usar `isActive=false` quando uma unidade sair de operacao, em vez de deletar.

6.2 Ward

Tabela de alas, setores ou unidades internas.

Colunas:

- `id`
- `hospitalId`
- `name`
- `type`
- `bedCount`
- `isActive`
- `version`
- `defaultSchedules`
- `createdAt`
- `updatedAt`

Relacionamentos:

- pertence a `Hospital` ;
- pode conter varios `Patient` .

Organizacao recomendada:

- padronizar nomes: `UTI Adulto` , `Pediatria` , `Clinica Medica` , etc.;
- usar `type` como categoria padronizada: `ICU` , `Ward` , `Emergency` , `Pediatrics` , `Other` ;
- preencher `defaultSchedules` para melhorar mapas, etiquetas e requisicoes.

6.3 Professional

Tabela de usuarios/profissionais.

Colunas:

- `id`
- `hospitalId`
- `name`
- `role`
- `registrationNumber`
- `cpf`
- `crn`
- `cpe`
- `managingUnit`
- `passwordHash`
- `isActive`
- `version`

- `createdAt`
- `updatedAt`

Roles usados:

- `general_manager`
- `local_manager`
- `nutritionist`
- `technician`

Relacionamentos:

- pertence opcionalmente a `Hospital` ;
- pode assinar/criar `Prescription` ;
- pode registrar `DailyEvolution` .

Seguranca:

- a senha nao deve ser salva em texto puro;
- o campo correto e `passwordHash` ;
- consulta de senha real nao e possivel nem desejavel;
- quando alguem esquece a senha, o correto e resetar a senha.

Organizacao recomendada:

- `registrationNumber` deve ser unico por hospital/perfil operacional;
- padronizar o papel de cada usuario;
- inativar usuarios desligados com `isActive=false` .

6.4 Patient

Tabela de pacientes.

Colunas:

- `id`
- `hospitalId`
- `wardId`
- `name`
- `bed`
- `recordNumber`
- `admissionDate`
- `birthDate`
- `gender`
- `weight`
- `height`
- `bmi`
- `diagnosis`
- `comorbidities`
- `allergies`
- `nutritionType`
- `targetKcal`
- `targetProtein`
- `targetVolume`
- `status`
- `observation`
- `consistency`

- `safeConsistency`
- `mealCount`
- `defaultSchedules`
- `isActive`
- `version`
- `createdAt`
- `updatedAt`

Status principais:

- `active`
- `discharged`
- `transferred`
- `deceased`

Relacionamentos:

- pertence a `Hospital` ;
- pertence opcionalmente a `Ward` ;
- possui varias `Prescription` ;
- possui varias `DailyEvolution` .

Organizacao recomendada:

- prontuario deve ir em `recordNumber` ;
- leito deve ir em `bed` ;
- ala/setor deve ir em `wardId` , nao apenas texto solto;
- metas nutricionais devem ficar em `targetKcal` , `targetProtein` , `targetVolume` e, se houver regra mais rica, no snapshot da prescricao/evolucao;
- status de alta/obito deve mudar o paciente de ativo para historico sem apagar dados.

6.5 Formula

Tabela de formulas, dietas enterais, suplementos e formulas infantis.

Colunas:

- `id`
- `hospitalId`
- `code`
- `name`
- `manufacturer`
- `type`
- `classification`
- `macronutrientComplexity`
- `ageGroup`
- `systemType`
- `formulaTypes`
- `administrationRoutes`
- `presentationForm`
- `presentations`
- `presentationDescription`
- `description`
- `billingUnit`
- `conversionFactor`

- `billingPrice`
- `density`
- `caloriesPerUnit`
- `proteinPerUnit`
- `proteinPct`
- `carbPerUnit`
- `carbPct`
- `fatPerUnit`
- `fatPct`
- `fiberPerUnit`
- `fiberType`
- `sodiumPerUnit`
- `potassiumPerUnit`
- `calciumPerUnit`
- `phosphorusPerUnit`
- `waterContent`
- `osmolality`
- `proteinSources`
- `carbSources`
- `fatSources`
- `fiberSources`
- `specialCharacteristics`
- `plasticG`
- `paperG`
- `metalG`
- `glassG`
- `isActive`
- `version`
- `createdAt`
- `updatedAt`

Uso:

- calculos nutricionais;
- prescricao;
- etiquetas;
- faturamento;
- relatorios de consumo;
- residuos reciclaveis.

Organizacao recomendada:

- `code` deve ser o codigo interno de faturamento/estoque;
- `name` deve ser nome do produto cadastrado;
- `description` deve trazer a descricao dietetica que aparece na evolucao/prescricao quando a equipe nao quer nome comercial;
- `type` deve separar `standard` , `oral-supplement` , `infant-formula` , etc.;
- `presentationForm` deve ser `liquido` ou `po` ;
- `billingUnit` deve ser `ml` , `g` ou `unit` ;
- `presentations` deve ser JSON padronizado com tamanhos, por exemplo `[1000]` .

6.6 Module

Tabela de modulos nutricionais e espessantes.

Colunas:

- id
- hospitalId
- code
- name
- manufacturer
- description
- presentationForm
- presentations
- conversionFactor
- density
- referenceAmount
- referenceTimesPerDay
- calories
- protein
- carbs
- fat
- sodium
- potassium
- calcium
- phosphorus
- fiber
- freeWater
- billingUnit
- billingPrice
- proteinSources
- carbSources
- fatSources
- fiberSources
- plasticG
- paperG
- metalG
- glassG
- isThickener
- isActive
- version
- createdAt
- updatedAt

Organizacao recomendada:

- usar `description` para texto dietetico generico;
- marcar espessante com `isThickener=true` ;
- preencher `billingUnit` e `billingPrice` para requisicao consolidada;
- separar modulo proteico, fibra, carboidrato, TCM e espessante por cadastro claro.

6.7 Supply

Tabela de insumos/material.

Colunas:

- id
- hospitalId
- code
- name
- type
- category
- description
- billingUnit
- capacityMl
- unitPrice
- isBillable
- plasticG
- paperG
- metalG
- glassG
- isActive
- version
- createdAt
- updatedAt

Tipos/categorias importantes:

- bottle
- set
- feeding-bottle
- baby-bottle
- pump-set
- gravity-set
- bolus-set
- hydration-water

Organizacao recomendada:

- frascos e equipos devem estar aqui, nao em `Formula` ;
- usar `capacityMl` para frascos;
- usar `isBillable=false` quando o insumo nao deve entrar no faturamento;
- manter `category` consistente, porque o faturamento automatico depende dela.

6.8 Prescription

Tabela principal de prescricao nutricional.

Colunas:

- id
- hospitalId
- patientId
- patientName
- patientRecord
- patientBed

- patientWard
- professionalId
- professionalName
- therapyType
- systemType
- feedingRoute
- infusionMode
- infusionRateMLH
- infusionDropsMin
- infusionHoursPerDay
- equipmentVolume
- hydrationVolume
- hydrationSchedules
- totalCalories
- totalProtein
- totalCarbs
- totalFat
- totalFiber
- totalVolume
- totalFreeWater
- nursingTimeMinutes
- nursingCostTotal
- materialCostTotal
- totalCost
- enteralDetails
- oralDetails
- parenteralDetails
- payloadSnapshot
- status
- statusReason
- statusChangedAt
- statusChangedBy
- startDate
- endDate
- notes
- version
- createdAt
- updatedAt

Uso:

- esta e a tabela central do app;
- guarda totais calculados e snapshots;
- relaciona paciente, profissional, formulas, modulos e insumos.

Campos JSON/texto importantes:

- enteralDetails
- oralDetails
- parenteralDetails
- payloadSnapshot
- hydrationSchedules

Hoje esses campos ficam como texto JSON. Funciona, mas dificulta relatorios SQL profundos. Para analítico avancado, o ideal e migrar para `Json` no Prisma/Postgres.

Organizacao recomendada:

- manter `Prescription` como cabecalho;
- manter itens em tabelas filhas;
- usar `status` para historico;
- nunca apagar prescricoes antigas;
- usar `startDate` / `endDate` para periodo de validade;
- guardar snapshot completo para auditoria.

6.9 PrescriptionFormula

Itens de formula dentro de uma prescricao.

Colunas:

- `id`
- `prescriptionId`
- `formulaId`
- `volume`
- `timesPerDay`
- `schedules`

Organizacao recomendada:

- uma linha por formula prescrita;
- `schedules` deve ser JSON array de horarios;
- para formula em po, `volume` hoje pode representar gramas dependendo do contexto, entao a regra de unidade deve ficar clara no snapshot/detalhes.

6.10 PrescriptionModule

Itens de modulo dentro de uma prescricao.

Colunas:

- `id`
- `prescriptionId`
- `moduleId`
- `amount`
- `timesPerDay`
- `schedules`
- `unit`

Organizacao recomendada:

- `amount` + `unit` devem sempre estar preenchidos;
- `unit` preferencialmente `g` ou `ml` ;
- horarios como JSON array.

6.11 PrescriptionSupply

Insumos associados a prescricao.

Colunas:

- `id`
- `prescriptionId`

- supplyId
- quantity

Organizacao recomendada:

- usar para insumos explicitamente vinculados;
- para frasco/equipo automatico, avaliar se e melhor gerar somente em faturamento ou persistir historico aqui.

6.12 PrescriptionStatusEvent

Historico de mudanca de status da prescricao.

Colunas:

- id
- prescriptionId
- fromStatus
- toStatus
- reason
- changedBy
- effectiveDate
- createdAt

Organizacao recomendada:

- registrar cancelamento, suspensao, alta, obito, conclusao;
- nao sobrescrever historico;
- usar para auditoria assistencial.

6.13 DailyEvolution

Acompanhamento diario.

Colunas:

- id
- hospitalId
- patientId
- prescriptionId
- professionalId
- date
- prescribedVolume
- infusedVolume
- infusionPercentage
- proteinPrescribed
- proteinInfused
- oralKcal
- oralProtein
- enteralKcal
- enteralProtein
- parenteralKcal
- parenteralProtein
- nonIntentionalKcal
- tneGoals
- tneInterruptions
- unintentionalCalories

- `gastricResidualVolume`
- `bowelMovements`
- `vomitingEpisodes`
- `bloodGlucose`
- `weight`
- `notes`
- `version`
- `createdAt`
- `updatedAt`

Uso:

- compara prescrito versus recebido;
- alimenta graficos de acompanhamento;
- registra interrupcoes;
- permite analisar deficit energetico/proteico.

Organizacao recomendada:

- uma linha por paciente por dia por contexto de acompanhamento;
- definir regra clara de data operacional: se a equipe registra um dia antes para dieta do dia seguinte, o sistema deve gravar `date` como data assistencial alvo, nao como timestamp de digitacao;
- migrar `tneGoals`, `tneInterruptions` e `unintentionalCalories` para `Json`;
- criar regra unica para fuso America/Sao_Paulo e evitar alternancia entre data atual/dia anterior.

6.14 AppSettings

Configuracoes da unidade.

Colunas:

- `id`
- `hospitalId`
- `nutritionistCostHour`
- `nurseCostHour`
- `technicianCostHour`
- `waterCostLiter`
- `energyCostKwh`
- `defaultSignatures`
- `labelSettings`
- `nursingCosts`
- `indirectCosts`
- `version`
- `createdAt`
- `updatedAt`

Organizacao recomendada:

- uma linha por hospital;
- custos devem ser revisados por periodo;
- assinaturas e conservacao de etiqueta devem ficar aqui;
- campos JSON deveriam ser `Json`.

6.15 RolePermission

Permissoes por perfil.

Colunas:

- `id`
- `hospitalId`
- `role`
- `permissionKey`
- `allowed`
- `version`
- `createdAt`
- `updatedAt`

Organizacao recomendada:

- usar como matriz de permissao por hospital;
- manter `permissionKey` padronizado com `src/lib/permissions.ts` ;
- evitar permissoes duplicadas.

6.16 AppTool

Ferramentas e links clinicos exibidos no app.

Colunas:

- `id`
- `hospitalId`
- `code`
- `name`
- `category`
- `description`
- `link`
- `isActive`
- `version`
- `createdAt`
- `updatedAt`

Organizacao recomendada:

- usar para catalogo de ferramentas clinicas;
- separar ferramentas internas de links externos;
- manter `code` unico por hospital.

6.17 IdempotencyRecord

Controle para evitar duplicidade em reenvios.

Colunas:

- `id`
- `key`
- `method`
- `path`
- `requestHash`
- `statusCode`
- `responseBody`
- `createdAt`
- `updatedAt`

Uso:

- quando o app offline reenvia uma operacao, o backend pode reconhecer a chave e evitar gravar duplicado;
- muito importante para fila offline.

Organizacao recomendada:

- criar rotina de limpeza para registros antigos, por exemplo manter 7 a 30 dias;
- indexar e monitorar crescimento.

7. Relacionamentos principais

Hospital

- > Ward
- > Professional
- > Patient
- > Formula
- > Module
- > Supply
- > AppSettings
- > RolePermission
- > AppTool

Patient

- > Prescription
- > DailyEvolution

Prescription

- > PrescriptionFormula -> Formula
- > PrescriptionModule -> Module
- > PrescriptionSupply -> Supply
- > PrescriptionStatusEvent
- > DailyEvolution

As relacoes mais importantes para relatorios sao:

- Patient.hospitalId ;
- Patient.wardId ;
- Prescription.patientId ;
- Prescription.hospitalId ;
- PrescriptionFormula.prescriptionId ;
- PrescriptionFormula.formulaId ;
- PrescriptionModule.prescriptionId ;
- PrescriptionModule.moduleId ;
- DailyEvolution.patientId ;
- DailyEvolution.prescriptionId ;
- DailyEvolution.date .

8. Indices existentes

O banco ja possui indices importantes:

- Patient(hospitalId, status, wardId) ;
- Patient(hospitalId, name) ;

- `Patient(hospitalId, recordNumber) ;`
- `Prescription(patientId, therapyType, status) ;`
- `Prescription(hospitalId, status) ;`
- `Prescription(startDate, endDate) ;`
- `DailyEvolution(hospitalId, date) ;`
- `DailyEvolution(patientId, date) ;`
- `DailyEvolution(prescriptionId, date) ;`
- `Formula(hospitalId, isActive, type) ;`
- `Formula(hospitalId, code) ;`
- `Formula(hospitalId, name) ;`
- `Supply(hospitalId, isActive, type) ;`
- `Supply(hospitalId, category) ;`
- `Supply(hospitalId, code) ;`
- `Professional(hospitalId, isActive, role) ;`
- `Professional(hospitalId, registrationNumber) ;`
- `Ward(hospitalId, isActive, name) ;`
- `RolePermission(hospitalId, role, permissionKey) unico;`
- `AppTool(hospitalId, code) unico;`
- `IdempotencyRecord(key) unico;`
- `IdempotencyRecord(createdAt) .`

Recomendacoes futuras de indices:

- `Prescription(hospitalId, therapyType, status, startDate) ;`
- `PrescriptionFormula(formulaId) ;`
- `PrescriptionModule(moduleId) ;`
- `PrescriptionSupply(supplyId) ;`
- `DailyEvolution(hospitalId, patientId, date) ;`
- se relatorios por ala forem pesados, manter `patientWard` normalizado ou juntar por `Patient.wardId` .

9. Pontos fortes da modelagem atual

- Ha separacao clara entre cabecalho de prescricao e itens.
- O banco ja tem `hospitalId` na maioria das tabelas relevantes.
- Ha controle de status para paciente e prescricao.
- Ha versionamento por registro.
- Ha fila offline com idempotencia.
- Ha tabelas para custo, residuos, materiais, formulas e modulos.
- Ha metricas e observabilidade ja conectadas a Prometheus/Grafana.
- O schema esta pronto para crescer para multi-hospital.

10. Problemas e riscos atuais

10.1 Validacao continua da origem dos dados

O banco atual contem dados, mas o app tambem possui cache offline e fallbacks locais. O risco principal nao e "banco vazio"; o risco real e confundir a origem de um dado exibido na tela.

Risco:

- uma tela pode exibir dado vindo do cache IndexedDB enquanto o backend ainda nao recebeu a sincronizacao;
- um usuario pode acreditar que salvou no servidor quando a operacao ficou apenas na fila offline;

- relatorios podem divergir temporariamente se existirem pendencias locais nao sincronizadas;
- dados de sessao e unidade selecionada em `localStorage` podem mascarar qual hospital esta sendo consultado;
- estatisticas estimadas do Postgres podem sugerir `0` registros mesmo quando existem dados reais.

Acao recomendada:

- mostrar no app quando ha fila offline pendente;
- exibir origem/estado do dado em telas criticas: `sincronizado` , `pendente` , `falhou` ;
- usar contagens via Prisma/API para auditoria, nao somente `pg_stat_user_tables` ;
- criar rotina de conciliacao diaria entre `pendingOperations` e o PostgreSQL;
- manter backup automatico do volume PostgreSQL e exportacao operacional em arquivo.

10.2 Muitos campos JSON como texto

Campos como:

- `enteralDetails` ;
- `oralDetails` ;
- `parenteralDetails` ;
- `payloadSnapshot` ;
- `hydrationSchedules` ;
- `tneGoals` ;
- `tneInterruptions` ;
- `unintentionalCalories` ;
- `defaultSignatures` ;
- `labelSettings` ;
- `nursingCosts` ;
- `indirectCosts` .

Hoje sao `String / text` com JSON serializado.

Funciona para a aplicacao, mas reduz qualidade analitica.

Recomendacao:

- migrar para tipo Prisma `Json` ;
- criar validadores Zod para cada estrutura;
- documentar schema de cada JSON;
- criar views SQL que extraem campos relevantes para relatorio.

10.3 Duplicidade entre snapshot e tabelas filhas

`Prescription` guarda totais e detalhes em JSON, enquanto `PrescriptionFormula` e `PrescriptionModule` guardam itens normalizados.

Isso e aceitavel se a regra for clara:

- tabelas filhas = fonte para relatorios estruturados;
- snapshot JSON = auditoria do estado da tela/calculo no momento da prescricao.

Sem essa regra, pode haver divergencia.

10.4 Unidade de medida precisa ficar explicita

Em formulas liquidas, `volume` representa mL.

Em formulas em po, pode representar g.

Risco:

- relatorio pode mostrar po em mL;

- requisicao pode consolidar unidade errada;
- etiqueta pode confundir volume total com quantidade de po.

Recomendacao:

- adicionar `unit` tambem em `PrescriptionFormula` ;
- ou separar `amount` e `finalVolumeMl` ;
- para dietas diluidas, gravar:
- `powderAmountG` ;
- `baseWaterMl` ;
- `finalVolumeMl` ;
- `billingQuantity` ;
- `billingUnit` .

10.5 Data operacional precisa de regra unica

Ja houve duvida se acompanhamento entra no dia atual, dia anterior ou amanha.

Regra recomendada:

- `createdAt` : quando foi digitado;
- `date` : data assistencial a que aquele acompanhamento se refere;
- `targetDeliveryDate` : data da dieta/entrega quando for diferente;
- nunca inferir data operacional apenas pelo horario do computador sem mostrar ao usuario.

Para lactario:

- requisicao feita em 08/06 pode gerar horarios de 09/06;
- isso deve estar explicitamente no documento e no banco.

10.6 Backup precisa ser rotina oficial

Como os dados ficam no PostgreSQL local, formatar a maquina sem backup perde a base.

Ja existem scripts:

- `infra/postgres/scripts/backup.ps1` ;
- `infra/postgres/scripts/restore.ps1` ;
- `infra/postgres/scripts/install-daily-backup-task.ps1` ;
- `infra/postgres/scripts/run-daily-backup.ps1` .

Recomendacao:

- agendar backup diario;
- salvar em pasta fora do projeto;
- copiar para nuvem/HD externo;
- testar restore mensalmente.

11. Como organizar os dados daqui para frente

11.1 Camada mestre/cadastros

Manter como cadastros mestres:

- `Hospital` ;
- `Ward` ;
- `Professional` ;
- `Formula` ;

- `Module` ;
- `Supply` ;
- `RolePermission` ;
- `AppSettings` ;
- `AppTool` .

Regra:

- cadastro mestre nao deve ser recriado varias vezes;
- deve ser atualizado e versionado;
- registros antigos ficam `isActive=false` .

11.2 Camada assistencial

Manter como dados assistenciais:

- `Patient` ;
- `Prescription` ;
- `PrescriptionFormula` ;
- `PrescriptionModule` ;
- `PrescriptionSupply` ;
- `PrescriptionStatusEvent` ;
- `DailyEvolution` .

Regra:

- nunca apagar historico assistencial;
- status encerra ciclos;
- toda prescricao precisa estar ligada a paciente e hospital;
- evolucao deve estar ligada a paciente e, quando possivel, a prescricao.

11.3 Camada operacional/faturamento

Hoje o faturamento e gerado a partir das prescricoes, formulas, modulos, insumos e horarios.

Para producao forte, criar tabelas de historico operacional:

- `BillingRequest` ;
- `BillingRequestItem` ;
- `DeliveryProtocol` ;
- `DeliveryProtocolItem` ;
- `LabelPrintBatch` ;
- `LabelPrintItem` .

Por que:

- hoje o documento pode ser gerado, mas nem tudo fica persistido como evento;
- para auditoria, e melhor saber quem gerou, quando gerou, qual filtro usou e quais itens saíram;
- permite reimprimir documento antigo identico.

11.4 Camada analitica

Criar views/materialized views:

- `vw_prescription_daily_items` ;
- `vw_consumption_by_product` ;
- `vw_consumption_by_patient_day` ;
- `vw_cost_by_ward` ;

- `vw_waste_by_product` ;
- `vw_infusion_goal_attainment` ;
- `vw_billing_requisition_items` .

Essas views devem transformar JSON/texto em colunas analíticas.

11.5 Camada de auditoria/sincronizacao

Manter e fortalecer:

- `IdempotencyRecord` ;
- `version` em tabelas principais;
- `createdAt` ;
- `updatedAt` ;
- `statusChangedAt` ;
- `PrescriptionStatusEvent` .

Adicionar futuramente:

- `AuditLog` ;
- `SyncDevice` ;
- `OfflineSyncBatch` ;
- `ErrorEvent` persistente, além das métricas.

12. Proposta de normalizacao futura

12.1 Migrar campos JSON texto para JSONB

Prisma:

```
enteralDetails Json?
oralDetails Json?
parenteralDetails Json?
payloadSnapshot Json?
tneGoals Json?
tneInterruptions Json?
unintentionalCalories Json?
```

PostgreSQL:

```
ALTER TABLE "Prescription"
  ALTER COLUMN "enteralDetails" TYPE jsonb USING "enteralDetails"::jsonb;
```

Antes disso, precisa garantir que todos os valores sejam JSON válido.

12.2 Criar unidade explicita em `PrescriptionFormula`

Adicionar:

```
unit String?
finalVolumeMl Float?
powderAmountG Float?
dilutionWaterMl Float?
billingQuantity Float?
billingUnit String?
```

Beneficio:

- acaba ambiguidade entre po em gramas e formula liquida em mL;
- melhora etiqueta;
- melhora requisicao;
- melhora relatorio gerencial.

12.3 Criar tabelas de documentos gerados

Exemplo:

```
model BillingRequest {
  id String @id @default(uuid())
  hospitalId String
  wardId String?
  startDate DateTime
  endDate DateTime
  therapyFilter String
  selectedTimes Json
  generatedBy String?
  createdAt DateTime @default(now())
}

model BillingRequestItem {
  id String @id @default(uuid())
  requestId String
  patientId String?
  productType String
  productId String?
  productName String
  quantity Float
  unit String
  unitPrice Float?
  subtotal Float?
}
```

13. Como consultar os dados hoje

Saude do backend e banco

```
Invoke-WebRequest -UseBasicParsing http://localhost:3000/health/ready
```

Esperado:

```
{"status":"ready","database":"ok"}
```

Ver tabelas existentes

```
docker exec enterall-postgres psql -U enterall -d enterall_smart_rx -c "\dt"
```

Ver tamanho do banco

```
docker exec enterall-postgres psql -U enterall -d enterall_smart_rx -c "SELECT  
pg_size_pretty(pg_database_size('enterall_smart_rx'));"
```

Ver contagem por tabela

No PowerShell, por causa de aspas em tabelas Prisma com maiúsculas, prefira criar um arquivo `.sql` ou usar Prisma Studio.

Opcoes:

```
cd server  
npx prisma studio
```

ou:

```
docker exec -it enterall-postgres psql -U enterall -d enterall_smart_rx
```

Dentro do `psql` :

```
SELECT count(*) FROM "Patient";  
SELECT count(*) FROM "Prescription";  
SELECT count(*) FROM "DailyEvolution";
```

14. Como importar dados reais

Se houver backup JSON no formato esperado pelo script:

```
cd C:\Users\igorp\Documents\enterall-smart-rx\server  
npm.cmd run backup:import -- C:\caminho\backup.json
```

O script `server/scripts/importBackup.ts` espera:

```
{  
  "version": 1,  
  "exportedAt": "2026-06-10T00:00:00.000Z",  
  "data": {  
    "hospitals": [],  
    "wards": [],  
    "professionals": [],  
    "patients": [],  
    "formulas": [],  
    "modules": [],  
    "supplies": [],  
    "prescriptions": [],  
    "dailyEvolution": [],  
    "settings": {}  
  }  
}
```

Se quiser apenas catalogo de formulas/modulos do codigo:


```
cd C:\Users\igorp\Documents\enterall-smart-rx\server
npm.cmd run sync:catalog
```

Mas isso nao cria pacientes, prescricoes, evolucoes nem usuarios reais.

Se quiser dados ficticios de demonstracao:

```
cd C:\Users\igorp\Documents\enterall-smart-rx\server
npx prisma db seed
```

Nao rode seed em producao sem confirmar, porque ele cria dados de exemplo.

15. Backup recomendado

Backup manual:

```
powershell -ExecutionPolicy Bypass -File infra\postgres\scripts\backup.ps1
```

Restore:

```
powershell -ExecutionPolicy Bypass -File infra\postgres\scripts\restore.ps1 -BackupPath
C:\caminho\arquivo.dump
```

Agendamento:

```
powershell -ExecutionPolicy Bypass -File infra\postgres\scripts\install-daily-backup-task.ps1
```

Recomendacao de destino:

- pasta local fora do projeto;
- OneDrive/Google Drive institucional;
- HD externo;
- retencao minima de 30 dias;
- backup mensal permanente.

16. Como deixar "nivel producao"

Prioridade alta:

1. Importar/restaurar dados reais no PostgreSQL. 2. Garantir usuario gestor local ativo em `Professional`. 3. Ativar backup diario e testar restore. 4. Criar tunnel nomeado Cloudflare, nao quick tunnel temporario. 5. Migrar campos JSON texto para JSONB. 6. Persistir documentos de faturamento/etiquetas emitidos. 7. Definir regra unica de data operacional. 8. Criar dashboards de qualidade de dados.

Prioridade media:

1. Criar views analiticas. 2. Criar rotina de limpeza de `IdempotencyRecord`. 3. Criar auditoria de alteracoes criticas. 4. Melhorar constraints de unicidade por hospital. 5. Criar testes automatizados de sync offline.

Prioridade baixa:

1. Integrar dados de estoque externo. 2. Integrar custo real de compras. 3. Criar data warehouse separado se o volume crescer muito.

17. Checklist de qualidade dos dados

Rodar periodicamente:

- pacientes ativos sem hospital;
- pacientes ativos sem leito;
- prescricoes ativas sem paciente ativo;
- prescricoes sem itens;
- formulas sem codigo;
- formulas sem preco;
- modulos sem unidade de faturamento;
- insumos faturaveis sem preco;
- evolucoes sem prescricao;
- evolucoes com data futura indevida;
- registros offline pendentes por mais de 24h;
- `IdempotencyRecord` antigo demais.

Exemplos SQL:

```
SELECT count(*) FROM "Patient" WHERE "hospitalId" IS NULL;
SELECT count(*) FROM "Patient" WHERE status = 'active' AND (bed IS NULL OR bed = '');
SELECT count(*) FROM "Prescription" WHERE status = 'active' AND "patientId" NOT IN (SELECT id FROM "Patient");
SELECT count(*) FROM "Formula" WHERE "isActive" = true AND (code IS NULL OR code = '');
SELECT count(*) FROM "Supply" WHERE "isBillable" = true AND ("unitPrice" IS NULL OR "unitPrice" = 0);
SELECT count(*) FROM "DailyEvolution" WHERE date > now() + interval '1 day';
```

18. Conclusao

O projeto ja tem uma arquitetura boa para operar em hospital:

- backend local;
- banco local;
- frontend web/PWA;
- fila offline;
- metricas;
- dashboards;
- scripts de backup;
- Prisma/schema versionado.

O que falta agora nao e estrutura: e populacao e governanca dos dados.

O ponto mais urgente e localizar/restaurar a base real, porque o PostgreSQL atual esta vazio. Depois disso, a organizacao ideal e tratar `Hospital`, `Ward`, `Professional`, `Formula`, `Module` e `Supply` como cadastros mestres; tratar `Patient`, `Prescription` e `DailyEvolution` como nucleo assistencial; e persistir faturamento/etiquetas como eventos operacionais auditaveis.

Com essas correcoes, o banco fica pronto para relatorio gerencial serio: consumo por produto, custo por ala, metas versus oferta real, residuos, requisicao por horario, historico assistencial e base de quantitativos para contrato.